

Contents

guide MLPerf Training Benchmark	1
0. 这篇 guide 怎么读	1
1. 当时的历史语境	2
2. 论文的核心问题	2
3. 原论文结构地图	3
4. 核心概念	3
5. 为什么不能只比 throughput	4
6. 软件多样性: 两个 momentum 公式为什么重要	4
7. 七个 benchmark 覆盖什么	4
8. Timing rules: 什么时候开始, 什么时候结束	5
9. Required runs: 随机性怎么处理	6
10. Hyperparameter rules	6
11. Submission and review	7
12. Reporting: division, category, scale	7
13. 一张机制图: MLPerf 如何把训练比较变公平	8
14. 本仓库的最小代码对应	8
15. 数学和指标	9
16. 实验证据链条	9
17. 局限性	10
18. 对现在 LLM infra 的意义	11
19. 本仓库学习路径	11
20. 常见误读	12
21. 用 AI agent 学这篇论文	12
22. 闭卷掌握检查	12

guide MLPerf Training Benchmark

原论文: MLPerf Training Benchmark
本地原文 PDF: [learning/infra-graduation/paper/01_mlperf_training_benchmark.pdf](#)
本地导读 PDF: [learning/infra-graduation/paper/guide_01_mlperf_training_benchmark.pdf](#)
作者: Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Matei Zaharia, and others
版本: arXiv v3, 2020-03-02, MLSys 2020

0. 这篇 guide 怎么读

MLPerf Training Benchmark 不是一篇提出新模型或新优化器的论文。它的贡献是把“怎么公平比较训练系统”这件事做成可执行规则。

如果你只记住一句话, 就是:

训练系统不能只比 step/s、FLOPS 或吞吐, 而要比从开始训练到达到规定质量目标的 end-to-end time-to-quality。

这篇论文对 infra 学习特别重要, 因为它把很多你前面学到的点统一起来:

- GPU 峰值算力不等于训练完成时间。

- 更大 batch 不一定减少总计算，因为可能需要更多 epoch 才收敛。
- 混合精度、算子替换、数据流水线、通信优化都可能改变最终质量。
- 同一模型在不同框架里可能并不完全等价。
- benchmark 必须同时定义任务、数据、模型、质量阈值、计时规则、提交规则和审查规则。

读这篇时不要把它当“排行榜说明书”，而要当“评测协议设计论文”。它的核心方法就是规则设计。

1. 当时的历史语境

传统计算领域已经有经典 benchmark，例如 LINPACK、SPEC、TPC。它们推动了超级计算、服务器和数据库系统的进步。机器学习系统也需要类似标准，因为训练加速器、框架、编译器、库、集群网络和存储都在快速增长。

但是深度学习训练比传统 benchmark 更难比较。

第一，优化会影响质量。传统 benchmark 里，一个矩阵乘法算得更快，只要答案正确就可以。深度学习里，低精度、数据增强、优化器、batch size、学习率 schedule、框架算子实现都可能让 step 更快，却让模型更晚达到目标精度，甚至永远达不到。

第二，训练是随机的。相同代码、相同超参、不同 seed，可能需要不同 epoch 才到目标质量。论文 Figure 1 展示 NCF 和 MiniGo 的 epochs-to-quality 有明显 run-to-run variation。

第三，软件和硬件太多样。你很难要求 CPU、GPU、TPU、不同框架、不同通信库都跑同一个 binary、同一个代码库、同一组超参。强行这么做会扼杀真实系统优化，也不代表用户实际会怎么跑。

第四，之前的 benchmark 各有缺口：

- DeepBench 这类 microbenchmark 便宜、隔离硬件，但不能代表完整训练。
- Fathom/TBD 这类 full-model throughput benchmark 覆盖模型，但不管是否达到质量。
- DAWNBench 开始使用 time-to-accuracy，并允许创新，但过于自由，导致硬件/软件平台之间直接比较困难。

MLPerf 的定位就是综合这些经验：用足够真实的 workload、end-to-end time-to-train 指标、参考实现、规则和开放提交，形成可比较又不完全扼杀创新的训练 benchmark。

2. 论文的核心问题

MLPerf 想解决的问题可以写成一句系统评测问题：

Given:

```
many hardware/software systems
many valid implementation choices
stochastic training dynamics
```

Need:

```
compare systems fairly
require final model quality
allow useful optimizations
make results reproducible
keep benchmark affordable
```

注意这个问题里有一个张力：

- 如果规则太死，大家只能跑参考代码，无法展示真实系统能力。
- 如果规则太松，大家可以换模型、换数据、换训练技巧，结果就无法说明硬件和软件栈谁更强。

MLPerf 的方法就是把这个张力拆成 closed division 和 open division, 再用 task suite、quality threshold、timing rule、required runs、submission review 来约束比较。

3. 原论文结构地图

建议按下面顺序读原文:

- Abstract: 先记住三个 ML 训练 benchmark 特有挑战。
- Section 1 Introduction: 读 MLPerf 五个高层目标和它从 DeepBench、Fathom、DAWNBench 继承了什么。
- Section 2 Background: 慢读四个挑战, 特别是优化影响质量、scale 影响 time-to-train、随机性、软件多样性。
- Section 3 MLPerf Training Benchmark: 读七个 benchmark、time-to-train metric、timing rules、quality thresholds、reference implementations。
- Section 4 Benchmarking Process: 读 submission/review、closed/open division、available/preview/research category、scale reporting、为什么没有单一总分。
- Section 5 Results: 读 v0.5 到 v0.6 的 performance/scaling 改进。
- Section 6 Conclusions: 读总结和 general lessons, 这里是论文最像“方法论”的地方。
- Appendix: 看 artifact checklist, 理解一个合规结果需要代码、日志、硬件软件描述和目标精度。

4. 核心概念

Time-to-train / time-to-quality

MLPerf 的核心指标是达到指定质量目标所需的训练时间。它不是平均 step time, 也不是 tokens/s, 也不是 GPU utilization。它把系统速度和模型质量绑在一起。

```
time_to_quality = first_time(validation_metric reaches target)
```

比如 ResNet-50 v1.5 不是比每秒多少 images, 而是比达到规定 Top-1 accuracy 需要多少时间。

Quality threshold

每个 benchmark 都有接近当时 state-of-the-art 的质量目标。目标不能太低, 否则早期训练噪声太大, 也无法暴露低精度或大 batch 优化对最终质量的影响。目标也不能过高, 否则多框架和多硬件系统很难稳定达到, 提交成本太高。

Reference implementation

MLPerf 给每个 benchmark 提供参考实现。参考实现不是性能 baseline, 不能拿它的速度评价系统。它的作用是精确定义模型、训练过程、数据质量和质量阈值。

Closed division

Closed division 适合直接比较系统性能。提交必须和参考实现保持等价: 模型、初始化、优化器/训练 schedule、数据遍历都要符合规则。只能改允许的超参。

Open division

Open division 鼓励创新, 可以换模型结构、优化过程或数据增强, 但仍必须使用同一数据集和同一质量 metric。它更像“解决同一任务最快能做到什么”, 而不是严格硬件/软件等价比较。

Required runs and trimmed mean

为处理随机性, MLPerf 要求多次 timing runs。vision task 要 5 次, 其他 task 要 10 次。报告时丢掉最快和最慢, 取中间结果的算术平均。

```
reported_time = mean(sorted(run_times)[1:-1])
```

这不是统计学完美方案，但工程上简单、可审查、能减少偶然快/慢 run 的影响。

5. 为什么不能只比 throughput

论文 Section 2.1 是全篇最重要的动机。它反复说明: training throughput 和 time-to-solution 不是同一个东西。

一个优化可能让每一步更快，但改变学习动态:

- 低精度训练早期 loss 看起来正常，后期 accuracy 才分叉。
- 大 batch 提高硬件利用率，却可能需要更多 epoch 到达目标质量。
- 不同 optimizer 实现数值上很接近，但在学习率变化时会有收敛差异。
- 数据增强或 padding 顺序不同，会让模型输入分布发生细微变化。

论文举了 ResNet-50 的例子: batch size 4K 时大约 64 epochs 能达到 74.9% Top-1, batch size 16K 可能需要超过 80 epochs, 计算量增加约 30%。所以一个系统 “step/s 更高” 并不自动意味着 “更快训练完”。

6. 软件多样性: 两个 momentum 公式为什么重要

论文用 SGD with momentum 说明框架差异。看起来都是 momentum，但实现可能不同。

Caffe 风格可以写成:

```
m_t = alpha * m_{t-1} + eta_t * grad_t
w_t = w_{t-1} - m_t
```

TensorFlow/PyTorch 风格可以写成:

```
m_t = alpha * m_{t-1} + grad_t
w_t = w_{t-1} - eta_t * m_t
```

如果 learning rate η_t 固定，两者差别可能很小；但如果 learning rate schedule 会变化，两者不再完全等价。大 batch 训练本来就靠 learning rate schedule 维持收敛，所以这些看似底层的框架差异会影响 time-to-quality。

这就是 MLPerf 必须定义 “数学等价” 和允许超参范围的原因。

7. 七个 benchmark 覆盖什么

MLPerf v0.5 训练套件包含七个任务，目标是覆盖 vision、language、recommendation、reinforcement learning，并覆盖不同 compute motif。

```
Image classification
  dataset: ImageNet
  model: ResNet-50 v1.5
  target: 74.9% Top-1 accuracy
```

```
Object detection, lightweight
  dataset: COCO 2017
  model: SSD-ResNet-34
  target: 21.2 mAP
```

Instance segmentation and object detection, heavyweight

dataset: COCO 2017
model: Mask R-CNN
target: 37.7 box min AP and 33.9 mask min AP

Translation, recurrent

dataset: WMT16 EN-DE
model: GNMT
target: 21.8 SacreBLEU

Translation, nonrecurrent

dataset: WMT17 EN-DE
model: Transformer
target: 25.0 BLEU

Recommendation

dataset: MovieLens-20M
model: NCF
target: 0.635 HR@10

Reinforcement learning

dataset/task: 9x9 Go
model: MiniGo
target: 40.0% professional move prediction

这七个任务不是为了覆盖所有 ML，而是为了覆盖不同系统压力：

- ResNet-50: 经典图像分类，卷积、batch norm、数据增强。
- SSD/Mask R-CNN: 检测和分割，ROI、NMS、高分辨率图像、更多 memory pressure。
- GNMT: RNN，长序列和循环依赖。
- Transformer: attention 和 dense matmul。
- NCF: embedding table 和不规则 memory access。
- MiniGo: 自我对弈产生数据，RL 训练和大量 forward pass。

对 LLM infra 学习者来说，最该学的是 “suite design”：一个 benchmark 套件要覆盖不同瓶颈，而不是只选一个自己擅长的 workload。

8. Timing rules: 什么时候开始，什么时候结束

MLPerf 的 timing rule 体现了一个重要原则：计时范围要接近真实训练，但也要排除不代表训练系统能力的噪声。

计时开始：

- 系统第一次接触 training 或 validation data。

计时结束：

- validation metric 达到规定 quality target。

排除项：

- System initialization。例如集群队列、管理员策略、节点诊断，这些更像运维环境差异。

- Model creation and initialization。某些框架会编译图，公共数据集训练时间短时编译占比不合理，所以允许排除最多 20 分钟模型创建时间。
- Data reformatting。例如把图片转成 TFRecord/RecordIO/LMDB。它通常一次执行、多次复用。

但有一个关键限制：不能把训练时的数据处理或数据增强偷挪到 reformatting 阶段。例如不能提前保存不同 crop 后的图片来绕过训练期间 augmentation 成本。

Allowed outside timing:

```
one-time format conversion
cluster/system startup
limited model creation
```

Must remain inside timing:

```
training-time preprocessing
data augmentation
validation quality evaluation
training loop until target quality
```

9. Required runs: 随机性怎么处理

MLPerf 不只跑一次。规则是：

- Vision tasks: 5 runs。
- Other tasks: 10 runs。
- 每次 run 都必须达到 target。
- 排序后丢掉最快和最慢。
- 报告中间 runs 的平均时间。

为什么 vision 只要 5 次，其他要 10 次？论文说这是基于参考实现行为选择的：vision tasks 需要保证同系统 90% entries 在 5% 内；其他 tasks 用 10 runs 保证 90% entries 在 10% 内。

这个规则把随机性从“大家口头解释”变成了审查规则。它也提醒你：在训练系统里，一个漂亮 run 不等于稳定能力。

10. Hyperparameter rules

MLPerf 允许调一部分超参，因为系统规模不同，batch size 和学习率 schedule 必须适配。但它限制可调空间，因为 benchmark 的目标不是比谁有更多算力做超参搜索。

论文 Table 2 里列出可修改项。可以概括为：

- 所有 SGD 类任务: batch size 和 learning-rate schedule parameters。
- ResNet-50 / SSD: maximum samples per training patch。
- Mask R-CNN: image candidates 数量。
- GNMT: learning-rate decay function、learning rate、decay start、decay interval、warmup function、warmup steps。
- Transformer: Adam 或 Lazy Adam、learning rate、warmup steps。
- NCF: Adam 或 Lazy Adam、learning rate、beta1、beta2。

此外还有 hyperparameter borrowing: 在 review 期间，一个提交者可以借用另一个提交者某个 benchmark 的超参重新提交，但不能改硬件或软件。论文说前两轮里这种 borrowing 确实改善了若干提交，说明相近规模下超参具有一定可迁移性。

11. Submission and review

一个 MLPerf submission 包含:

- System description: 硬件节点数、处理器/加速器数量和类型、每节点存储、网络互连、操作系统、库版本。
- Training-session logs: 结构化时间戳、重要阶段、质量评估记录、超参选择。
- Code and libraries: 用于复现训练 sessions 的代码和依赖。

提交会经过 peer review。发现不合规，提交者可以修复后重新提交。这个流程说明 MLPerf 不是“发一个数字”，而是“发一个可审查实验包”。

```
submission package
|
+-- system description
+-- training logs
+-- code and libraries
+-- quality evidence
|
v
MLPerf rule review
|
+-- pass: publish result
+-- fail: explain noncompliance, allow resubmission
```

12. Reporting: division, category, scale

每个结果有几个标签。

Division:

- Closed: 同 workload 等价，适合直接系统比较。
- Open: 鼓励算法和系统共设计，适合展示创新解法。

Category:

- Available: 硬件可购买或云上可租，软件可普遍使用和支持。
- Preview: 组件会在 60 天内或下一轮前达到 available 条件。
- Research: 原型系统，或规模超过 available 配置的实验系统。

System type:

- On-premises。
- Cloud。

Scale reporting:

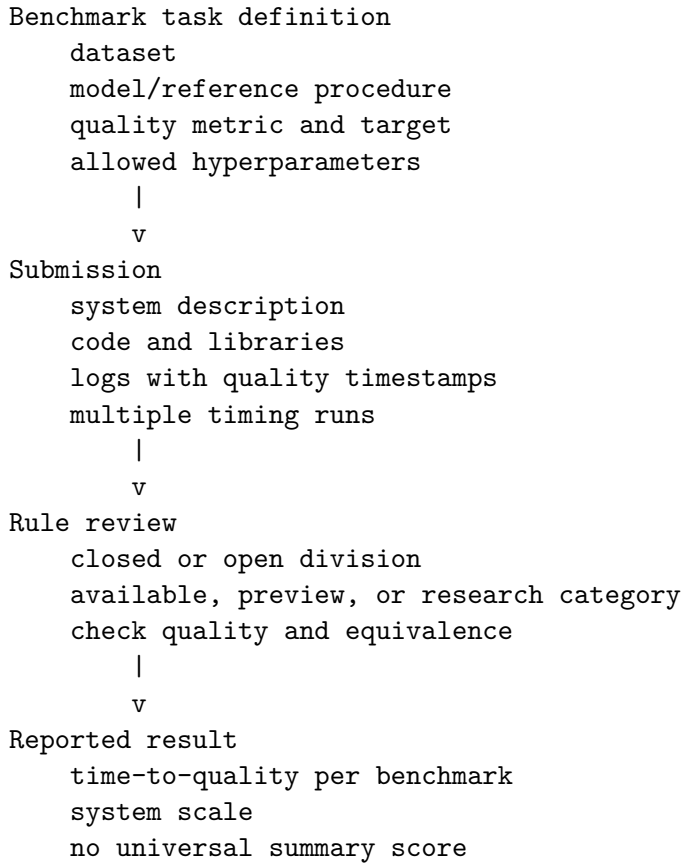
- 早期报告包含处理器/加速器数量和类型。
- 对 on-prem，后续需要 power measurement specification。
- 对 cloud，论文提出 cloud-scale metric，用主机处理器、内存、加速器数量和类型近似成本。

MLPerf 没有给整套 benchmark 一个单一总分。原因有两个:

- 不同用户对任务权重不同，没有 universally representative weighting。
- 提交者可能不报所有 benchmark，单一总分会变得不稳定或不公平。

这点很值得学习: 好 benchmark 不一定要把所有东西压成一个分数。很多时候分项结果更诚实。

13. 一张机制图: MLPerf 如何把训练比较变公平



这张图就是论文方法。它没有新 loss，但它定义了“什么样的训练结果算可比”。

14. 本仓库的最小代码对应

这次新增:

- learning/infra-graduation/src/eval/mlperf_original_minimal.py

它把论文规则压成 toy implementation:

- BenchmarkTask: 定义 dataset、model、metric、target、required_runs。
- TrainingRun: 保存每个 seed 的质量事件。
- reported_time_to_quality: 检查每次 run 达标，丢 fastest/slowest，求均值。
- Submission: 表示 closed/open submission 的规则字段。
- validate_submission: 检查 dataset、metric、code、logs、closed division 等价性和允许超参。
- speedup: 对 v0.5 到 v0.6 的时间改进做简单比值。

最小示例:

```
resnet = MLPERF_V05_TASKS[0]
runs = [
    _make_run(0, 120, resnet.target),
    _make_run(1, 125, resnet.target),
    _make_run(2, 128, resnet.target),
    _make_run(3, 130, resnet.target),
    _make_run(4, 200, resnet.target),
```

```
]
```

```
reported = reported_time_to_quality(resnet, runs)
```

```
# Drop 120 and 200, then average 125, 128, 130.
```

```
assert round(reported, 2) == 127.67
```

这段对应论文的 required runs 和 trimmed mean。你可以把它当成 MLPerf scoring 的最小骨架。

15. 数学和指标

Time-to-quality

t_i = first elapsed time in run i where $\text{metric}_i(t)$ reaches target

如果 metric 越高越好:

t_i = min t such that $\text{metric}_i(t) \geq \text{target}$

如果 metric 越低越好:

t_i = min t such that $\text{metric}_i(t) \leq \text{target}$

Reported time

```
times = sorted([t_1, t_2, ..., t_n])
```

```
reported = mean(times[1:-1])
```

这里 $n = 5$ for vision tasks, $n = 10$ for other tasks.

Speedup

```
speedup = old_time_to_quality / new_time_to_quality
```

如果 v0.5 最快 16-chip 结果是 100 分钟, v0.6 是 76.9 分钟:

```
speedup = 100 / 76.9 = 1.30
```

Cost-to-quality, 本仓库扩展

论文主要报告 time 和 scale, 不给统一 cost score。本仓库可以扩展一个工程视角:

```
dollars_to_quality
```

```
    = time_hours * cluster_cost_per_hour
```

这不是 MLPerf 原始主指标, 但对你做 infra 选型很有用。一个系统 time-to-quality 快, 但如果用 10 倍硬件, TCO 未必更好。

16. 实验证据链条

MLPerf 论文的实验重点不是证明某个硬件最好, 而是证明 MLPerf 作为 benchmark 能推动进步。

Evidence 1: Figure 1 说明训练随机性

NCF 和 MiniGo 即使只改 random seed, 也会出现不同 epochs-to-quality。MiniGo 甚至在固定 seed 下也有明显变动。这支撑 required runs 和 trimmed mean 规则。

Evidence 2: Figure 2 说明早期质量阈值不稳定

ResNet-50 五个 seed 的 accuracy 曲线在前 30 epochs 波动更大，后期更稳定。质量阈值如果定得太早，benchmark 会被随机性支配，也不能暴露后期质量问题。

Evidence 3: Section 2.1.2 说明 scale 改变学习动态

ResNet-50 batch 4K 和 16K 的 epochs-to-quality 不同。大 batch 能提升系统利用率，但可能需要更多 epoch。这个证据直接反驳“只比 throughput”。

Evidence 4: Table 1 说明 suite 覆盖多样 workload

七个 benchmark 覆盖 vision、translation、recommendation、RL。系统不能只为一个卷积网络做优化就声称“训练系统很强”。

Evidence 5: Section 4 说明可审查性

提交需要系统描述、日志、代码和库，并经过 review。这个机制支撑 reproducibility，比单纯上传一个数字更可靠。

Evidence 6: Figure 3 说明 benchmark 推动软件栈改进

v0.5 到 v0.6 相隔约六个月，硬件大体不变。最快 16-chip entries 平均约 1.3x speedup，同时若干 quality targets 还提高了。这说明提交者在实现、软件栈和规则适配上快速进步。

Evidence 7: Figure 4 说明 scaling 改进

达到最快 time-to-solution 所需 chips 数量在 v0.6 更大，平均约 5.5x。这说明系统更能利用更大规模集群。它也提醒我们：最快结果常常来自扩大 scale，所以报告结果必须带系统规模。

Evidence 8: Conclusions 的 general lessons

论文总结了几个经验：

- Realistic dataset size 很重要。初始 NCF 数据集太小，可能完全驻留内存，不能代表真实 memory-system 行为。
- 对公共小数据集，启动时间不应压倒训练本身，所以计时要排除不代表大规模训练的 startup。
- 小超参变化能带来明显 performance 变化，但相近规模下超参有一定可移植性。
- 框架的优化器细节会影响收敛。

这些经验比具体 v0.5/v0.6 数字更值得记住。

17. 局限性

第一，benchmark suite 永远会过时。ML 模型和 workload 发展很快，所以 MLPerf 必须靠 working groups 持续更新。

第二，closed division 的公平性来自限制创新。它适合系统等价比较，但不一定代表用户愿意用的最强训练 recipe。

第三，open division 鼓励创新，但比较对象变得复杂。一个 open result 快，可能来自模型、数据增强、优化器、系统、硬件共同变化。

第四，公共数据集可能比真实工业数据小。论文特别提到推荐系统数据集的代表性问题；小数据集会低估数据加载和 memory-system 压力。

第五，time-to-quality 不等于 cost-to-quality。论文报告 scale，但没有统一成本总分。真实采购还要考虑 capex、opex、电力、利用率、云折扣、工程维护。

第六，required runs 仍然昂贵。完整训练多次对小团队不便宜，所以规则必须在统计稳定和可参与性之间折中。

18. 对现在 LLM infra 的意义

MLPerf Training 的精神对 LLM infra 仍然非常有用，即使具体任务已经演进。

LLM 训练里也有同样问题：

- tokens/s 更高不一定代表更快达到目标 loss 或 benchmark。
- FP8、量化 optimizer、sequence packing、data mixture、large batch 都可能改变收敛。
- checkpoint、data loader、network allreduce、optimizer sharding、pipeline bubble 都会影响 end-to-end time。
- 不同训练框架和 kernel 对数值路径有差异。
- 只报 GPU utilization 或 MFU，不能完全说明训练系统能力。

所以你以后看任何 LLM 训练报告，都应该问：

- 它的 quality target 是什么？
- 它从什么时候开始计时？
- 是否包含 data loading、eval、checkpoint？
- 跑了几次？随机性如何处理？
- 系统 scale 是多少？成本是多少？
- 是否公开代码、日志、配置和硬件细节？
- 它是在 closed-like 等价比较，还是 open-like recipe 创新？

这就是 MLPerf 论文真正教你的思维。

19. 本仓库学习路径

相关 lecture：

- learning/infra-graduation/lectures/01-grad-overview.md
- learning/infra-graduation/lectures/04-mlperf.md

相关代码：

- learning/infra-graduation/src/eval/mlperf_original_minimal.py
- learning/infra-graduation/src/eval/mlperf_mock.py
- learning/infra-graduation/src/sim/time_to_train.py
- learning/infra-graduation/src/sim/cost_model.py
- learning/infra-graduation/src/sim/topology_selector.py
- learning/infra-graduation/src/portfolio_v3.py

建议 30 到 60 分钟实验：

1. 运行测试：

```
.venv\Scripts\python.exe learning\infra-graduation\src\tests\test_all.py
```

2. 打开 mlperf_original_minimal.py。

3. 把 ResNet 的 5 次 run 从 [120, 125, 128, 130, 200] 改成 [120, 125, 128, 130, 1000]。

4. 观察 reported time 是否不受最慢 outlier 影响。

5. 新增一个 closed submission，把 model_equivalent=False，确认 validation fail。

6. 新增一个 open submission，允许 model_equivalent=False，确认 validation pass。

预期观察：

- Trimmed mean 可以降低极端 run 的影响，但不能解决系统性不稳定。
- Closed division 保护等价比较。

- Open division 保护创新空间。
- Benchmark 规则本身就是系统设计的一部分。

20. 常见误读

误读 1: MLPerf 比的是硬件峰值。

不对。MLPerf Training 比的是达到质量目标的 end-to-end training time，并报告系统规模。

误读 2: Throughput 高就一定 MLPerf 快。

不对。大 batch 或低精度可能改变收敛，需要完整训练到 target 才能判断。

误读 3: Reference implementation 是性能 baseline。

不对。参考实现主要定义模型和训练过程，不是优化到极致的性能实现。

误读 4: Closed division 更高级，open division 不重要。

不对。Closed 适合公平系统比较，Open 适合探索更强 recipe 和 co-design。它们回答不同问题。

误读 5: 一个总分最方便，所以一定最好。

不对。MLPerf 不给总分，是因为任务权重没有普适答案，且并非所有系统都适合所有 benchmark。

21. 用 AI agent 学这篇论文

推荐让 agent 用“评测协议审稿人”方式陪你学：

我正在读 MLPerf Training Benchmark。请不要泛泛总结。

一次只问我一个问题，并要求我把答案落到规则或代码上：

1. 为什么 step throughput 不能替代 time-to-quality?
2. closed division 和 open division 分别保护什么?
3. timing rule 排除了哪些东西，为什么?
4. required runs 和 trimmed mean 解决什么随机性问题?
5. 为什么 MLPerf 不给一个 suite-wide summary score?
6. 请让我在 mlperf_original_minimal.py 中改一个规则并预测测试结果。

另一个有用提示词：

请站在 benchmark 设计者角度，审查一个 LLM 训练报告。

按 MLPerf 的思路检查：quality target、timing boundary、runs、scale、reproducibility、closed/open 类别、cost-to-quality。

每指出一个问题，都要说明它可能怎样误导系统比较。

22. 闭卷掌握检查

1. MLPerf 论文说 ML training benchmark 有哪三个独特挑战?
2. 为什么优化 throughput 可能让 time-to-quality 变差?
3. ResNet batch 4K 和 16K 的例子说明了什么?
4. Figure 1 和 Figure 2 分别支持哪条规则?
5. Closed division 要求哪些等价性? Open division 放松了什么?
6. MLPerf 什么时候开始计时，什么时候停止计时?
7. 哪些东西可以排除计时? 哪些训练时工作不能挪出去?
8. Vision tasks 和其他 tasks 分别需要几次 run? 报告值如何计算?
9. 为什么参考实现不是 performance baseline?

10. 一个 submission package 至少应该包含哪些东西?
11. 为什么 MLPerf 不给单一总分?
12. v0.5 到 v0.6 的 Figure 3/4 分别说明了什么?
13. 如果你要把 MLPerf 思想迁移到 LLM 预训练,你会定义哪些 quality targets 和 timing boundaries?
14. 在本仓库里, mlperf_original_minimal.py 哪个函数对应 trimmed mean? 哪个函数对应 closed/open rule review?

真正掌握的标志是: 你能把任何 “某系统训练更快” 的宣传, 拆成 quality、time、scale、rules、logs、code、cost 七个问题逐一追问。